

PucePark — Análisis de Diseño de Sistemas de Información

Proyecto Integrador P02 · PUCE TEC Sistema de gestión de parqueaderos universitarios (app móvil + backend de microservicios).

1. Requerimientos Funcionales (RF)

ID	Requerimiento	Actor	Prioridad
RF-01	Iniciar sesión con una única pantalla que diferencia estudiante/guardia según el rol del token (AWS Cognito).	Estudiante, Guardia	Alta
RF-02	Un usuario autenticado que no tenga cuenta en el sistema ve el mensaje “No estás registrado, ve a Secretaría”.	Estudiante	Media
RF-03	Completar el perfil (nombre y apellido, placa, permiso / cédula) mediante onboarding antes de usar el sistema.	Estudiante, Guardia	Alta
RF-04	Listar las zonas de parqueo con su disponibilidad (disponibles/ocupados).	Estudiante, Guardia	Alta
RF-05	Ver el mapa de puestos de una zona con estados por color (verde=disponible, amarillo=mi puesto, rojo=ocupado).	Estudiante, Guardia	Alta
RF-06	Ocupar un puesto disponible. El sistema impide ocupar más de uno a la vez.	Estudiante	Alta
RF-07	Liberar el puesto propio desde un botón fijo, sin desplazarse por la lista.	Estudiante	Alta
RF-08	Guardia: registrar entrada manual (forzar ocupación) indicando la placa del vehículo.	Guardia	Media
RF-09	Guardia: forzar la liberación de un puesto ocupado.	Guardia	Media
RF-10	Consultar el historial de parqueos del propio usuario.	Estudiante	Media
RF-11	Consultar estadísticas personales mensuales (horas, sesiones, racha).	Estudiante	Baja
RF-12	Consultar el ranking mensual de usuarios por horas acumuladas.	Estudiante, Guardia	Baja
RF-13	Editar el perfil (ícono de lápiz) y ver los datos personales.	Estudiante, Guardia	Media
RF-14	Administrar zonas y puestos (crear, actualizar, eliminar).	Admin	Media
RF-15	Restringir acciones según rol; un usuario sin permiso recibe 401/403.	Sistema	Alta
RF-16	Cerrar sesión y volver a iniciar sin residuos de sesión previa.	Estudiante, Guardia	Media

2. Requerimientos No Funcionales (RNF)

ID	Requerimiento	Cómo se cumple en PucePark
RNF-01 Seguridad	Autenticación y autorización basada en tokens.	JWT emitido por AWS Cognito; cada microservicio valida el token contra el JWKS del <i>issuer</i> (Resource Server). Roles por <code>cognito:groups</code> .
RNF-02 Escalabilidad	El sistema debe poder crecer por servicio.	Microservicios independientes (park-app, users-service) con base de datos por servicio ; servicios stateless detrás de nginx → replicables horizontalmente.
RNF-03 Disponibilidad	Recuperación ante fallos de contenedor.	<code>restart: unless-stopped</code> + <code>healthcheck</code> de las BDs en docker-compose; <code>depends_on: condition: service_healthy</code> .
RNF-04 Concurrencia	Evitar doble ocupación del mismo puesto.	Bloqueo pesimista (<code>findByIdWithPessimisticLock</code>) al ocupar.
RNF-05 Mantenibilidad	Código organizado y comprensible.	Arquitectura en capas (controller/service/repository) + DTOs/mappers/entities; una excepción por archivo + manejador global.
RNF-06 Portabilidad	Despliegue reproducible en cualquier entorno.	Todo dockerizado; punto de entrada único por nginx (puerto 80).
RNF-07 Usabilidad	Interfaz clara con validaciones.	App iOS con validación de formularios (placa, permiso, nombre y apellido), leyenda de colores, estados de carga/error.
RNF-08 Consistencia	Respuestas de error uniformes.	<code>GlobalExceptionHandler</code> con <code>ExceptionResponse { message, source }</code> y códigos HTTP adecuados (404/409/400/401/403).

3. Tabla de Casos de Uso

Caso de uso	Actor	Precondición	Flujo principal	Postcondición
CU-01 Iniciar sesión	Estudiante/Guardia	Tener credenciales PUCE	1) Ingresar usuario/clave 2) Cognito valida y emite JWT 3) La app lee el rol y navega a Zonas	Sesión activa con token y rol
CU-02 Completar perfil	Estudiante/Guardia	Sesión activa, perfil incompleto	1) Se muestra onboarding 2) Ingresar nombre+apellido, placa/cédula, permiso 3) Guardar	<code>Perfil complete=true</code> en users-service
CU-03 Ocupar puesto	Estudiante	Perfil completo, sin puesto activo	1) Elige zona 2) Toca un puesto disponible 3) Confirma "Ocupar"	Puesto en OCUPADO; registro en historial
CU-04 Liberar puesto	Estudiante	Tener un puesto activo	1) Toca "Liberar" 2) El sistema cierra la sesión de parqueo	Puesto en DISPONIBLE; <code>exit_date</code> registrado
CU-05 Forzar ocupación	Guardia	Rol GUARD, puesto disponible	1) Selecciona puesto 2) Ingresar placa 3) Registra entrada	Puesto OCUPADO a nombre "GUARDIA:usuario"
CU-06 Forzar liberación	Guardia	Rol GUARD, puesto ocupado	1) Selecciona puesto 2) Confirma liberación	Puesto DISPONIBLE
CU-07 Ver ranking mensual	Todos	Sesión activa	1) Abre pestaña Ranking 2) Selecciona mes	Lista ordenada por horas
CU-08 Administrar zonas	Admin	Rol ADMIN	CRUD de zonas/puestos	Catálogo actualizado

4. GitFlow en GitHub (criterio 1.2)

El equipo gestiona el código en **GitHub** con un flujo basado en ramas + Pull Requests:

Repositorios: - Backend (microservicios): `github.com/Thyago23/ae_2026_01_Taco_Cede-o_1462_PucePark` - Frontend (app iOS): `github.com/BryanTaco/PuceParkFront`

Se aplica el modelo **GitFlow clásico** (Vincent Driessen), el mismo enseñado en clase con el proyecto de referencia *Micromercado*.

Ramas de larga vida: - **main** — código en producción; solo recibe versiones estables (merges de `release/*` o `hotfix/*`). - **develop** — rama de integración; acumula las funcionalidades terminadas antes de una versión.

Ramas de apoyo (temporales): - **feature/HU-NN-descripcion** — una por Historia de Usuario; nace de `develop` y regresa a `develop` (ej. `feature/HU-06-ocupar-puesto`). *Convención tomada del ejemplo Micromercado* (`feature/scrum-XX-HU-YY-...`). - **release/x.y** — preparación de una versión; nace de `develop`, se prueba y se mergea a `main` y a `develop`. - **hotfix/x.y.z** — corrección urgente en producción; nace de `main` y regresa a `main` y a `develop`.

Flujo de una Historia de Usuario: 1. `git checkout develop && git pull` 2. `git checkout -b feature/HU-NN-descripcion` 3. Desarrollo + *commits* con prefijos convencionales (`feat:`, `fix:`, `refactor:`). 4. `git push -u origin feature/HU-NN-descripcion` 5. **Pull Request** hacia `develop`, revisión del equipo y *merge*. 6. Al completar el alcance, `release/x.y` desde `develop` → `main` (despliegue).



Convención de commits: mensajes en español con prefijo de tipo (`feat:`, `fix:`, `refactor:`, `docs:`) y co-autoría del equipo (`Co-Authored-By`).

Evidencia en GitHub: - Ambos repos tienen las ramas de larga vida **main** y **develop**. - Rama de HU: `feature/HU-01-documentacion-gitflow` → PR hacia `develop` (esta misma documentación). - Refactor a microservicios integrado por PR (perfiles en `users-service`, ranking sin *joins*, nginx con re-resolución). - Historial de *commits* con prefijos convencionales visible en cada repo.

5. Pruebas Unitarias (criterio 1.3)

El backend incluye pruebas de: - **Services** (lógica de negocio): casos válidos e inválidos, uso de mocks (Mockito) para repositorios. - **GlobalExceptionHandler**: cubre la rama del mensaje por defecto de cada excepción (para JaCoCo). - **Controllers** (MockMvc + JWT): endpoints públicos/privados y validación de roles.

Ejecución: `./gradlew test` (y cobertura con JaCoCo).

6. ADR — Architecture Decision Records (criterio 1.4)

ADR-001 · Arquitectura de microservicios con base de datos por servicio

- **Contexto:** el sistema maneja dos dominios distintos: parqueo (zonas/puestos/historial) e identidad de usuario (perfiles).
- **Decisión:** separar en dos microservicios (`park-app`, `users-service`), cada uno con su propia base PostgreSQL (`puce_park`, `puce_micro`).

- **Consecuencias:** (+) despliegue y escalado independiente, aislamiento de fallos; (–) no se pueden hacer *joins* entre servicios → se resuelve con denormalización (ver ADR-004).

ADR-002 · Autenticación centralizada con AWS Cognito (Resource Server)

- **Decisión:** delegar la identidad a un User Pool de Cognito; cada servicio valida el JWT por su cuenta con el JWKS del *issuer*, sin llamadas entre servicios.
- **Consecuencias:** (+) servicios desacoplados, escalable; (+) roles vía `cognito:groups`; (–) dependencia de un proveedor externo.

ADR-003 · nginx como reverse proxy (punto de entrada único)

- **Decisión:** exponer solo nginx en el puerto 80; enruta `/api/*` → park-app y `/users/*` → users-service. Los servicios quedan internos (`expose`).
- **Consecuencias:** (+) un solo punto de entrada, oculta la topología interna; se añadió `resolver 127.0.0.11` para resolver IPs al reconstruir contenedores.

ADR-004 · Denormalización del nombre en el historial para el ranking

- **Contexto:** el ranking (park-app) necesita mostrar el nombre del usuario, que vive en users-service; el patrón prohíbe *joins* entre BDs.
- **Decisión:** el cliente envía el `fullName` al ocupar; park-app lo guarda en `parking_history.display_name`; el ranking lo lee sin *joins*.
- **Consecuencias:** (+) respeta el aislamiento de servicios; (–) dato duplicado (aceptable, es un valor de solo lectura para el ranking).

ADR-005 · Priorización de requerimientos

- **Decisión:** se priorizó primero el núcleo (login, ver zonas, ocupar/liberar) por ser el objetivo principal; luego historial/ranking/perfil; por último administración.
- **Justificación:** entregar valor temprano al usuario final (estudiante) y asegurar los criterios de “objetivo principal” y “conexión a API con BD” antes que funciones secundarias.